Contents

1 Abstract **3**

2 Introduction **4**

3 Mechanical Design **4**

3.1 Robot Parameters 5

3.2 Coordinate System 5

4 Inverse Position Kinematics Analysis **6**

4.1 IPK Geometry 7

4.2 Law of Cosines 7

4.3 Active Joint Angles 8

4.4 IPK Algorithm 8

5 Forward Position Kinematics **8**

6 Workspace Analysis **9**

7 Velocity Kinematics **10**

7.1 Loop Closure Equations 10

7.2 Differentiation of the Loop Closures 11

7.3 Jacobian Matrices 11

8 Static Force Analysis **13**

8.1 Virtual Work Method 13

8.2 Free-Body Diagram Interpretation 14

9 Trajectory Generation **15**

9.1 Linear Trajectory 15

9.2 Circular Trajectory 15
9.3 Infinity Trajectory 16

10 Simscape Simulation **16**

11 Hardware Implementation **18**

11.1 Hardware Components 18
11.2 Pin Configuration 18
11.3 MATLAB-Arduino Serial Protocol 19
11.4 Angle-to-Step Conversion 20
11.5 Trajectory Execution Logic 21
11.6 Open-Loop Calibration 21

12 Results and Discussion **22**

13 Limitations **23**

14 Conclusion **23**

15 Future Work **23**

A MATLAB IPK Code **24**

B References **25**

1. Abstract

This project presents the mechanical design, kinematic analysis, simulation, and hardware implementation of a planar pantograph parallel robot used as a drawing mechanism. The robot consists of two actuated revolute joints and two passive distal links connected to a common end-effector point. The main objective is to control the end-effector position in the planar workspace and execute predefined drawing trajectories such as a straight line, circle, and infinity sign.

The report covers the mechanical structure, inverse position kinematics, forward position kinematics, velocity kinematics, static force analysis, Simscape simulation, MATLAB trajectory generation, and Arduino-based stepper motor control. A dedicated section is included for inverse position kinematics, where the desired end-effector position is converted into active motor angles. Simulation results are compared with the real hardware behavior, and the final system demonstrates the ability to execute drawing trajectories using a MATLAB-to-Arduino serial interface.

2. Introduction

Parallel manipulators are mechanisms in which the moving platform or end-effector is connected to the fixed base by multiple kinematic chains. Compared to serial manipulators, parallel mechanisms can provide higher stiffness, improved load distribution, and a compact structure. In this project, a planar pantograph parallel robot was designed and implemented as a drawing robot.

The purpose of the project is to analyze and control the motion of the pantograph robot so that the end-effector can follow predefined trajectories. The required work includes mechanical design, inverse and forward position kinematics, velocity kinematics, static force analysis, Simscape simulation, and real hardware execution using stepper motors controlled by Arduino.

The main objectives of this project are:

1. Design and model the pantograph parallel robot mechanically.
2. Derive the inverse position kinematics of the mechanism.
3. Verify the forward position kinematics.
4. Derive the velocity kinematics and Jacobian matrices.
5. Analyze the static force relation using virtual work and free-body diagram concepts.
6. Generate line, circle, and infinity trajectories.
7. Simulate the robot motion in Simscape.
8. Implement the motion on real hardware using MATLAB and Arduino.

3. Mechanical Design

The pantograph mechanism consists of four links connected by revolute joints. Links l_1 and l_2 are the proximal links connected directly to the two actuated base joints. Links l_3 and l_4 are the distal links connected to the common end-effector point P. The active joints are located at A_1 and A_2, while the joints at B_1 and B_2 are passive. The coordinate frame is placed at the midpoint O between the two active joints. The two base joints are separated by a distance d along the x-axis.

3.1. Robot Parameters

Table 1: Pantograph robot geometric parameters

Parameter	Description	Value
l_1	Left proximal link length	12.5 cm
l_2	Right proximal link length	12.5 cm
l_3	Left distal link length	16.6 cm
l_4	Right distal link length	16.6 cm
d	Distance between active joints	5.8 cm

All MATLAB hardware-control calculations were performed consistently in centimeters. When simulation trajectories were generated in millimeters, the values were converted before hardware execution.

3.2. Coordinate System

The fixed base frame origin O is located at the midpoint between the two active joints. Therefore, the two active joint locations are:

$$A_1 = \begin{bmatrix} -\frac{d}{2} \\ 0 \end{bmatrix} \tag{1}$$

$$A_2 = \begin{bmatrix} \frac{d}{2} \\ 0 \end{bmatrix} \tag{2}$$

The passive joint positions B_1 and B_2 are functions of the active motor angles θ_1 and θ_2:

$$B_1 = \begin{bmatrix} -\frac{d}{2} + l_1 \cos\theta_1 \\ l_1 \sin\theta_1 \end{bmatrix} \tag{3}$$

$$B_2 = \begin{bmatrix} \frac{d}{2} + l_2 \cos\theta_2 \\ l_2 \sin\theta_2 \end{bmatrix} \tag{4}$$

The end-effector position is defined as:

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \tag{5}$$

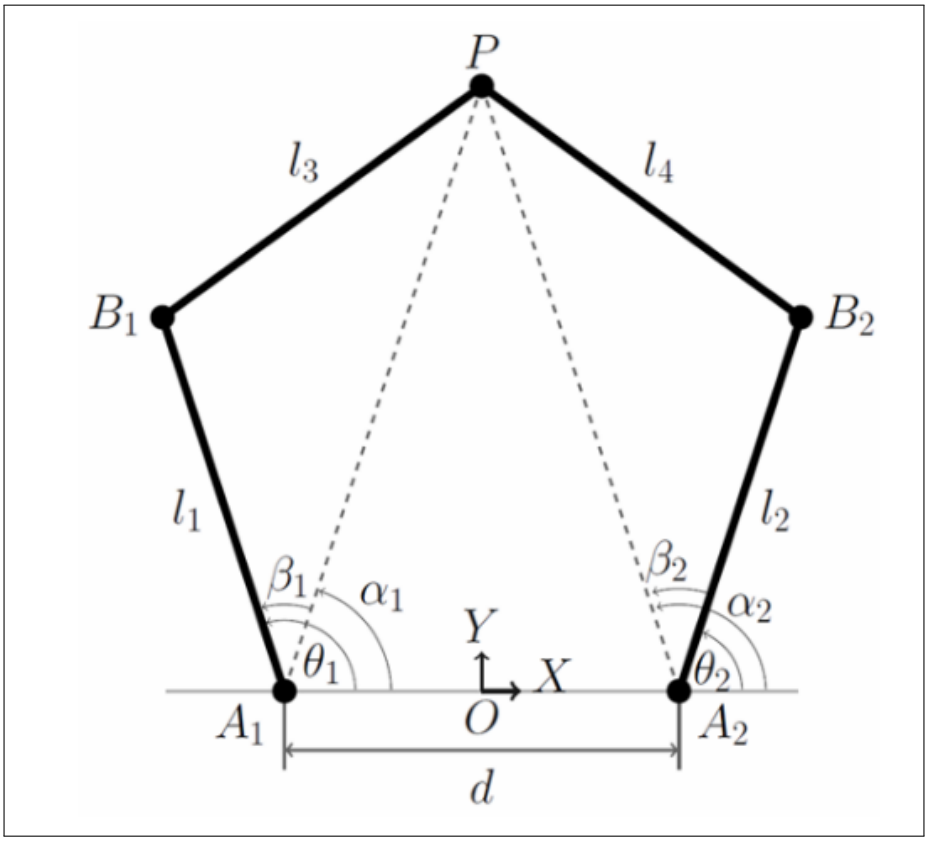


Figure 1: Pantograph robot geometry.

4. Inverse Position Kinematics Analysis

Inverse Position Kinematics (IPK) is used to determine the active joint angles required to place the end-effector at a desired position. For the pantograph robot, the input to IPK is the desired end-effector position:

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \tag{6}$$

and the output is the active joint vector:

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} \tag{7}$$

4.1. IPK Geometry

The distances from the active joints to the desired end-effector point are:

$$a_1 = \sqrt{\left(x + \frac{d}{2}\right)^2 + y^2} \tag{8}$$

$$a_2 = \sqrt{\left(x - \frac{d}{2}\right)^2 + y^2} \tag{9}$$

The angles α_1 and α_2 define the directions from the active joints to the end-effector point:

$$\alpha_1 = \tan^{-1}\left(\frac{y}{x + \frac{d}{2}}\right) \tag{10}$$

$$\alpha_2 = \tan^{-1}\left(\frac{y}{x - \frac{d}{2}}\right) \tag{11}$$

In MATLAB, the function `atan2(y,x)` is used instead of `atan(y/x)` to correctly handle all quadrants.

4.2. Law of Cosines

Using the law of cosines, the internal angles between the active links and the lines connecting the active joints to the end-effector are calculated.

For the left chain:

$$b_1 = \frac{l_1^2 + a_1^2 - l_3^2}{2 l_1 a_1} \tag{12}$$

$$\beta_1 = \cos^{-1}(b_1) \tag{13}$$

For the right chain:

$$b_2 = \frac{l_2^2 + a_2^2 - l_4^2}{2 l_2 a_2} \tag{14}$$

$$\beta_2 = \cos^{-1}(b_2) \tag{15}$$

4.3. Active Joint Angles

The selected assembly mode for the physical robot gives:

$$\theta_1 = \alpha_1 + \beta_1 \tag{16}$$

$$\theta_2 = \alpha_2 - \beta_2 \tag{17}$$

This assembly mode was selected because it corresponds to the physical front-half working configuration of the robot. Points outside the reachable workspace or outside the active joint limits are rejected before trajectory execution.

4.4. IPK Algorithm

The IPK algorithm follows these steps:

1. Define the desired end-effector position $P = [x, y]^T$.
2. Compute the distances a_1 and a_2.
3. Compute the direction angles α_1 and α_2.
4. Apply the law of cosines to compute β_1 and β_2.
5. Compute the active joint angles θ_1 and θ_2.
6. Check the joint limits and reject invalid configurations.

For real hardware execution, the Simscape angle offsets were not used. The Arduino receives the real IPK motor angles only.

5. Forward Position Kinematics

Forward Position Kinematics (FPK) calculates the end-effector position from the active joint angles. First, the passive joint positions B_1 and B_2 are calculated using the active

angles θ_1 and θ_2. Then, the end-effector point P is obtained as the intersection of two circles:

- Circle 1: center B_1, radius l_3
- Circle 2: center B_2, radius l_4

The constraints are:

$$|P - B_1| = l_3 \tag{18}$$

$$|P - B_2| = l_4 \tag{19}$$

Equivalently:

$$(P - B_1)^T (P - B_1) = l_3^2 \tag{20}$$

$$(P - B_2)^T (P - B_2) = l_4^2 \tag{21}$$

The circle intersection generally gives two mathematical solutions. The solution with the higher y-coordinate was selected because the robot operates in the front/top working assembly mode.

To verify the FPK and IPK consistency, a test point was passed into the IPK function to obtain θ_1 and θ_2. These angles were then passed into the FPK function. The resulting point matched the original point with negligible numerical error, confirming the correctness of the kinematic model.

6. Workspace Analysis

The reachable workspace was obtained by scanning a grid of x- and y-coordinates and applying the IPK function at each point. A point was accepted only if it satisfied the link-length reachability conditions and the active joint limits.

The front-half workspace was selected because the physical robot is intended to draw in front of the motor baseline. The joint limits were also used to avoid configurations close to mechanical interference or singular behavior.

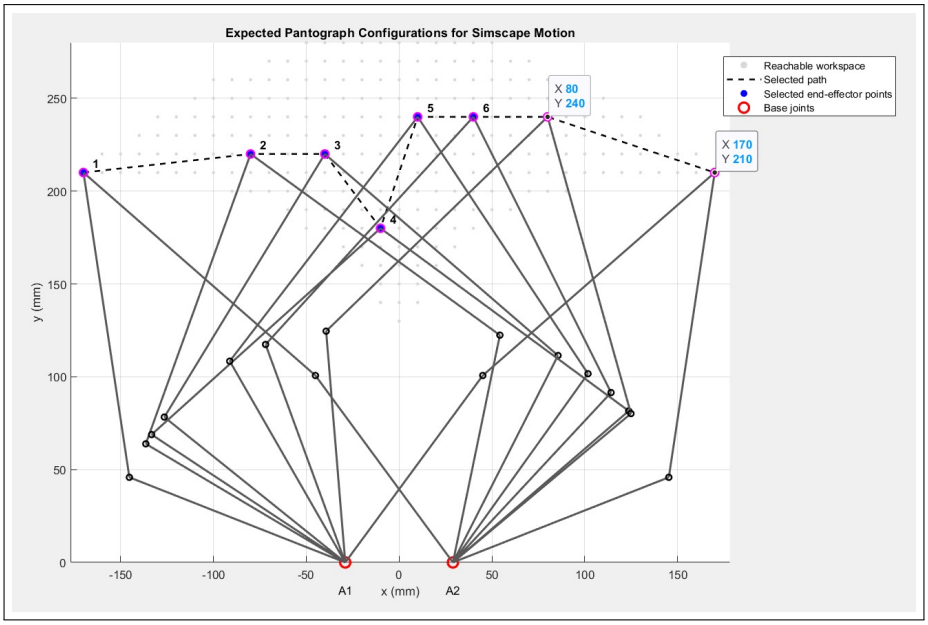


Figure 2: Reachable workspace generated in MATLAB.

7. Velocity Kinematics

Velocity kinematics relates the end-effector velocity to the active joint velocities. For parallel robots, the relation is obtained by differentiating the closed-loop kinematic constraints.

The active joint vector is:

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} \tag{22}$$

The end-effector variable vector is:

$$\chi = \begin{bmatrix} x \\ y \end{bmatrix} \tag{23}$$

7.1. Loop Closure Equations

For the left chain:

$$P - B_1 = l_3 \hat{s}_3 \tag{24}$$

For the right chain:

$$P - B_2 = l_4 \hat{s}_4 \tag{25}$$

where $\hat{s}_3$ and $\hat{s}_4$ are unit vectors along the passive links B_1P and B_2P.

7.2. Differentiation of the Loop Closures

For the left chain:

$$\dot{P} - \dot{B}_1 = l_3 \dot{\hat{s}}_3 \tag{26}$$

Since $\hat{s}_3$ is a rotating unit vector:

$$\dot{\hat{s}}_3 = \dot{\alpha}_3 (\hat{k} \times \hat{s}_3) \tag{27}$$

Therefore:

$$\dot{P} - \dot{B}_1 = l_3 \dot{\alpha}_3 (\hat{k} \times \hat{s}_3) \tag{28}$$

Dot multiplying by $\hat{s}_3^T$ eliminates the passive angular velocity because $\hat{s}_3$ is perpendicular to $(\hat{k} \times \hat{s}_3)$:

$$\hat{s}_3^T (\dot{P} - \dot{B}_1) = 0 \tag{29}$$

For the right chain:

$$\hat{s}_4^T (\dot{P} - \dot{B}_2) = 0 \tag{30}$$

7.3. Jacobian Matrices

The end-effector velocity is:

$$\dot{\chi} = \dot{P} = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \tag{31}$$

The active joint velocity vector is:

$$\dot{q} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \tag{32}$$

The passive link unit vectors are:

$$\hat{s}_3 = \begin{bmatrix} s_{3x} \\ s_{3y} \end{bmatrix} \tag{33}$$

$$\hat{s}_4 = \begin{bmatrix} s_{4x} \\ s_{4y} \end{bmatrix} \tag{34}$$

The velocity relation can be written as:

$$J_x \dot{\chi} = J_q \dot{q} \tag{35}$$

where:

$$J_x = \begin{bmatrix} s_{3x} & s_{3y} \\ s_{4x} & s_{4y} \end{bmatrix} \tag{36}$$

and:

$$J_q = \begin{bmatrix} l_1(-s_{3x}\sin\theta_1 + s_{3y}\cos\theta_1) & 0 \\ 0 & l_2(-s_{4x}\sin\theta_2 + s_{4y}\cos\theta_2) \end{bmatrix} \tag{37}$$

The general Jacobian of the parallel manipulator is:

$$\dot{q} = J\dot{\chi} \tag{38}$$

where:

$$J = J_q^{-1} J_x \tag{39}$$

This Jacobian gives the active joint velocities required to achieve a desired end-effector velocity.

8. Static Force Analysis

Static force analysis relates the actuator torques to the force applied at the end-effector. The analysis assumes static equilibrium and neglects inertial effects.

8.1. Virtual Work Method

The virtual work principle states that the total virtual work at static equilibrium is zero:

$$\delta W = 0 \tag{40}$$

The virtual work done by the actuators is:

$$\delta W_{actuators} = \tau^T \delta q \tag{41}$$

The virtual work done by the end-effector force is:

$$\delta W_{external} = F^T \delta \chi \tag{42}$$

Therefore:

$$0 = \tau^T \delta q - F^T \delta \chi \tag{43}$$

Using the Jacobian relation:

$$\delta q = J \delta \chi \tag{44}$$

Substitute into the virtual work equation:

$$0 = \tau^T J \delta \chi - F^T \delta \chi \tag{45}$$

$$0 = (\tau^T J - F^T) \delta \chi \tag{46}$$

Since the equation must hold for any virtual displacement $\delta \chi$:

$$F^T = \tau^T J \tag{47}$$

Taking the transpose:

$$F = J^T \tau \tag{48}$$

Thus, the actuator torques required to produce a desired end-effector force are:

$$\tau = J^{-T} F \tag{49}$$

For the planar pantograph:

$$F = \begin{bmatrix} F_x \\ F_y \end{bmatrix} \tag{50}$$

and:

$$\tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} \tag{51}$$

8.2. Free-Body Diagram Interpretation

The same relation can be understood using a free-body diagram. The end-effector force F is balanced by the internal forces in the two passive links. Let T_3 and T_4 be the forces along the two passive link directions $\hat{s}_3$ and $\hat{s}_4$.

Force equilibrium at the end-effector gives:

$$T_3 \hat{s}_3 + T_4 \hat{s}_4 = F \tag{52}$$

In matrix form:

$$\begin{bmatrix} s_{3x} & s_{4x} \\ s_{3y} & s_{4y} \end{bmatrix} \begin{bmatrix} T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} F_x \\ F_y \end{bmatrix} \tag{53}$$

Taking moments about the active joints gives:

$$\tau_1 = T_3 l_1(\cos\theta_1 s_{3y} - \sin\theta_1 s_{3x}) \tag{54}$$

$$\tau_2 = T_4 l_2(\cos\theta_2 s_{4y} - \sin\theta_2 s_{4x}) \tag{55}$$

This gives the same physical result as the virtual work method.

9. Trajectory Generation

The robot was required to execute three task-space trajectories: a straight line, a circle, and an infinity sign. Each trajectory was first generated as a sequence of end-effector points in the $x-y$ plane. The IPK function was then applied to convert every point into active joint angles.

9.1. Linear Trajectory

A straight-line trajectory is generated using linear interpolation:

$$P(t) = P_{start} + \alpha(P_{end} - P_{start}) \tag{56}$$

where:

$$0 \leq \alpha \leq 1 \tag{57}$$

9.2. Circular Trajectory

A circular trajectory is generated using:

$$x = c_x + r\cos\theta \tag{58}$$

$$y = c_y + r\sin\theta \tag{59}$$

where (c_x, c_y) is the circle center and r is the radius.

9.3. Infinity Trajectory

Two versions of the infinity trajectory were used. The Simscape visualization used a workspace-centered infinity path with timeseries signals. The real hardware version used a batch angle trajectory sent to Arduino. The final hardware implementation was adjusted so that the physical robot starts from the same point as the Simscape trajectory.

The Simscape-style infinity equation is:

$$x = c_x + \frac{s \cos t}{1 + \sin^2 t} \tag{60}$$

$$y = c_y + \frac{s \sin t \cos t}{1 + \sin^2 t} \tag{61}$$

where s is the scale of the infinity sign.

10. Simscape Simulation

The mechanical model was simulated in Simscape Multibody. The CAD model was imported into Simscape, and the active revolute joints were actuated using angle timeseries inputs generated from MATLAB. The desired infinity trajectory was first converted from task-space points into joint angles using the IPK function. These joint angles were then converted into Simscape-compatible signals.

The Simscape model required angle offsets because the Simscape revolute joint frames did not have the same zero-angle convention as the theoretical MATLAB model. Therefore, the simulation signals were generated as:

$$\theta_{1,sim} = \theta_{1,matlab} + \frac{\pi}{2} \tag{62}$$

$$\theta_{2,sim} = \theta_{2,matlab} + \frac{\pi}{2} \tag{63}$$

These offsets were used only for simulation. They were not sent to the real hardware.

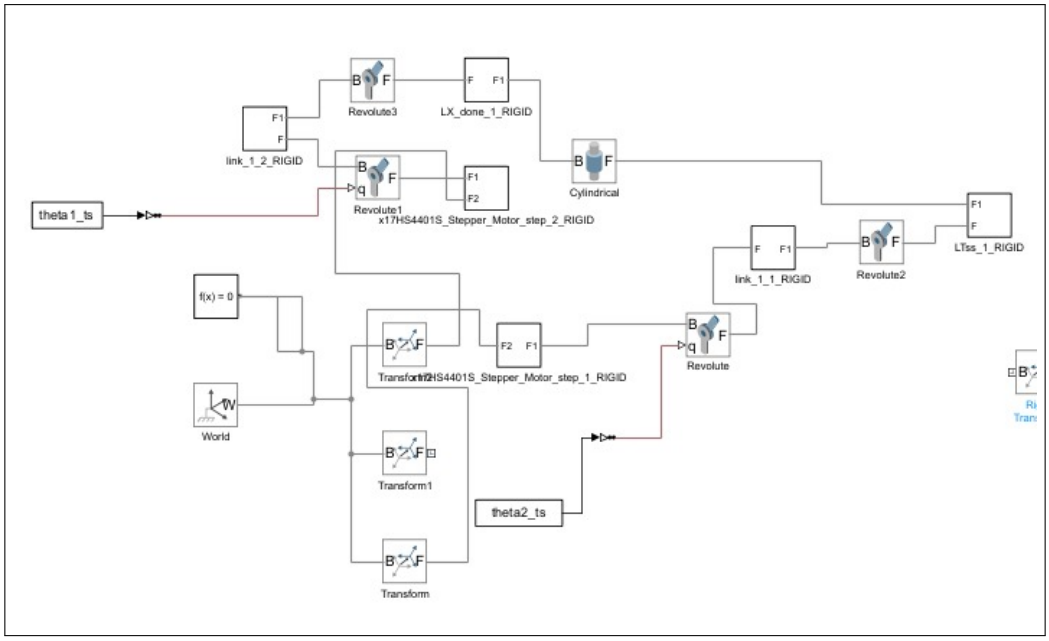


Figure 3: Simscape model of the pantograph mechanism.

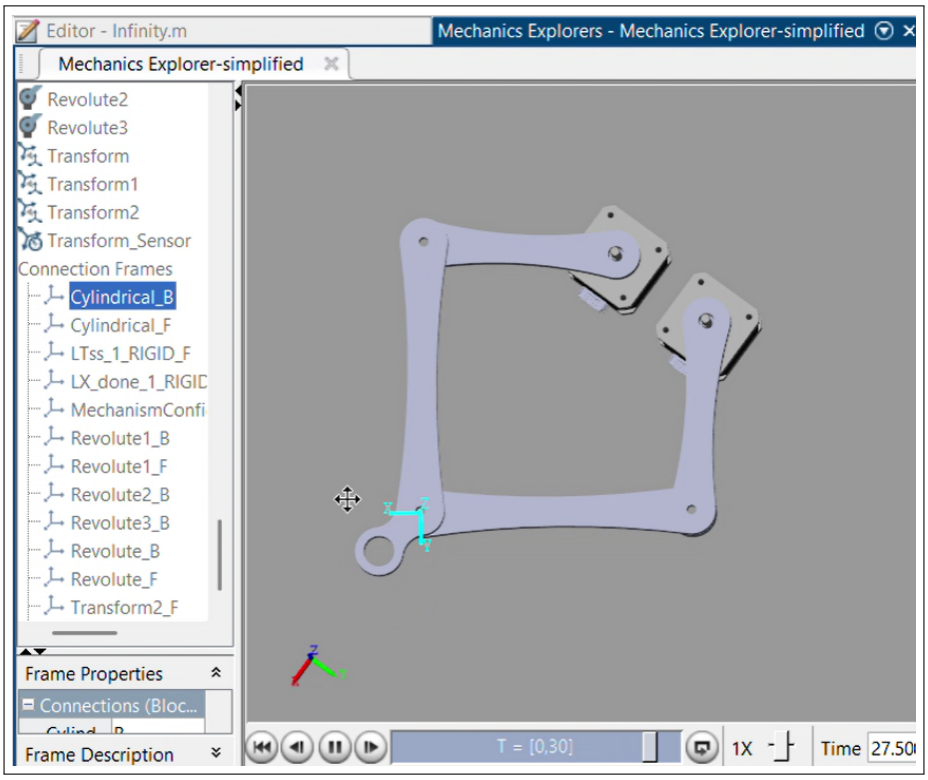


Figure 4: Simscape animation of the infinity sign trajectory.

11. Hardware Implementation

The real pantograph robot was implemented using two stepper motors driven by TMC2209 stepper motor drivers and controlled by an Arduino board. MATLAB was responsible for generating the task-space trajectory, applying the inverse position kinematics, and sending the resulting active joint angle trajectory to Arduino through serial communication. The Arduino was responsible for real-time execution by converting the received joint angles into stepper motor steps and generating the required STEP/DIR pulses.

This architecture was selected because MATLAB and USB serial communication are not suitable for generating precise real-time step pulses directly. Therefore, MATLAB was used for high-level trajectory and kinematic calculations, while Arduino handled the low-level real-time motor control.

11.1. Hardware Components

Table 2: Hardware components used in the real robot

Component	Function
Arduino board	Main controller for motor commands
Two NEMA 17 stepper motors	Actuated revolute joints
Two TMC2209 stepper drivers	Stepper motor driving using STEP/DIR signals
External power supply	Power source for the stepper motors
Pantograph mechanical structure	Planar parallel drawing mechanism
Pen holder	End-effector for drawing trajectories
MATLAB	Trajectory generation and inverse kinematics calculation
Arduino IDE	Uploading the low-level stepper motor control code

11.2. Pin Configuration

The TMC2209 drivers were controlled using STEP and DIR pins. The enable pin was shared between the two drivers. The microstepping pins were configured to set the drivers to 1/16 microstepping.

Table 3: Arduino pin configuration

Signal	Arduino Pin	Description
STEP1	3	Step pulse signal for motor 1
DIR1	2	Direction signal for motor 1
MS1_1	7	Microstepping pin 1 for driver 1
MS2_1	6	Microstepping pin 2 for driver 1
STEP2	9	Step pulse signal for motor 2
DIR2	10	Direction signal for motor 2
MS1_2	5	Microstepping pin 1 for driver 2
MS2_2	8	Microstepping pin 2 for driver 2
EN_PIN	4	Shared enable pin for both drivers

The TMC2209 enable pin is active-low, so the drivers are enabled by setting the enable pin to LOW. The microstepping mode was set by writing both microstepping pins HIGH:

$$MS1 = HIGH, \qquad MS2 = HIGH \tag{64}$$

This configuration gives 1/16 microstepping.

11.3. MATLAB-Arduino Serial Protocol

The communication between MATLAB and Arduino was performed using serial communication. MATLAB sends the complete trajectory to Arduino before execution. The serial command format is:

```
HOME,theta1,theta2
BEGIN,N
P,theta1,theta2
P,theta1,theta2
...
END
RUN
```

The purpose of each command is explained in Table 4.

Table 4: MATLAB-Arduino serial communication protocol

Command	Function
`HOME,theta1,theta2`	Synchronizes Arduino's internal motor position with the physical home position
`BEGIN,N`	Tells Arduino that N trajectory points will be sent
`P,theta1,theta2`	Sends one target angle pair to be stored on Arduino
`END`	Indicates that all trajectory points have been sent
`RUN`	Starts executing the stored trajectory

The `HOME` command does not move the motors. It only tells Arduino that the robot is currently placed physically at the known home configuration. This is important because the system is open-loop and does not use encoders. Therefore, the robot must be manually placed at the required home position before running the trajectory.

11.4. Angle-to-Step Conversion

The joint angles calculated by MATLAB are sent to Arduino in degrees. Arduino converts each angle into a target number of motor steps. For a standard stepper motor with 200 full steps per revolution and 1/16 microstepping, the number of microsteps per revolution is:

$$N_{steps/rev} = 200 \times 16 = 3200 \tag{65}$$

Since one full revolution is 360°, the number of steps per degree is:

$$\text{steps per degree} = \frac{200 \times 16 \times G}{360} \tag{66}$$

where G is the gear ratio. In this project, the gear ratio was assumed to be:

$$G = 1 \tag{67}$$

Therefore:

$$\text{steps per degree} = \frac{200 \times 16}{360} = 8.8889 \tag{68}$$

The target step position for each motor is calculated as:

$$\text{target steps} = \theta_{deg} \times 8.8889 \tag{69}$$

For example, if MATLAB sends a target angle of 90°, the corresponding motor step position is:

$$90 \times 8.8889 = 800 \text{ steps} \tag{70}$$

11.5. Trajectory Execution Logic

Instead of sending one command and waiting for the robot to stop at every point, MATLAB sends the full trajectory to Arduino first. Arduino stores all target angle points, converts them into step positions, and then executes the complete motion after receiving the `RUN` command.

This batch execution method reduces jerky motion because the Arduino controls the timing of the step pulses locally without waiting for MATLAB after every point. The execution flow is:

1. MATLAB generates the desired task-space trajectory.
2. MATLAB applies IPK to convert each point into θ_1 and θ_2.
3. MATLAB sends the complete angle trajectory to Arduino.
4. Arduino converts the angles into motor step positions.
5. Arduino generates synchronized STEP/DIR pulses for both motors.
6. The robot executes the desired drawing trajectory.

11.6. Open-Loop Calibration

Since no encoder feedback was used, the system works as an open-loop stepper motor system. This means Arduino only knows the motor position based on the number of steps commanded, not from actual measured position feedback.

Therefore, before each run, the robot must be manually placed at the correct physical home position. MATLAB calculates the joint angles corresponding to this home position and sends them using the `HOME` command. Arduino then stores these angles as its current internal step position.

Any error in manual homing causes an offset in the real drawing. Also, if the motors skip steps because of high speed, high friction, or excessive load, Arduino will not detect the error. This is one of the main limitations of the current hardware implementation.

12. Results and Discussion

The IPK and FPK were first verified numerically in MATLAB. The IPK function was used to calculate the active joint angles for a selected end-effector position, and the FPK function was then used to recover the end-effector position. The resulting error was negligible, confirming that the position kinematic model was consistent.

The Simscape simulation successfully demonstrated the pantograph motion following the desired trajectory. The infinity sign was generated in task space, converted to joint-space angles, and applied to the active revolute joints as timeseries inputs.

The hardware implementation successfully executed line and infinity trajectories. The circle trajectory required adjustment of the circle center and radius to remain within the reachable workspace. The main hardware challenges were open-loop stepper control, manual home calibration, mechanical friction, pen contact force, and small jerks due to discretized stepper movement.

The final hardware system used batch trajectory execution instead of sending one point at a time. This reduced stop-start motion because the Arduino stored the full trajectory before execution.



Figure 5: Hardware drawing result.

13. Limitations

The system is open-loop because no encoders or external position feedback were used. Therefore, the robot assumes that the motors do not skip steps and that the physical home position is correctly set before each run. Any error in the manual home calibration appears as an offset in the final drawing. Mechanical backlash, friction, and pen-paper contact also affect drawing accuracy.

Another limitation is that Simscape and real hardware use different angle conventions. Simscape required angle offsets due to revolute joint frame orientation, while the real robot used the actual IPK motor angles directly.

14. Conclusion

The project successfully developed and implemented a planar pantograph parallel robot for drawing applications. The mechanical design was modeled, and the kinematic equations were derived and verified. A dedicated IPK algorithm was developed to convert desired end-effector positions into active joint angles. FPK was used to verify the kinematic consistency, while velocity kinematics and static force analysis were derived using the Jacobian.

The robot was simulated in Simscape and implemented on real hardware using MATLAB and Arduino. MATLAB generated the desired trajectories and calculated the required motor angles, while Arduino executed the motion using stepper motor control. The final system demonstrated the ability to draw predefined trajectories, including a line and an infinity sign.

15. Future Work

Future improvements include adding encoder feedback for closed-loop position control, improving mechanical stiffness, reducing backlash, adding automatic homing sensors, optimizing trajectory timing, and controlling pen pressure. A camera-based feedback system could also be used to compare the drawn trajectory with the desired path and correct errors automatically.

A. MATLAB IPK Code

```
function theta = IPK_pantograph(q, robot)
    x = q(1);
    y = q(2);

    l1 = robot.l1;
    l2 = robot.l2;
    l3 = robot.l3;
    l4 = robot.l4;
    d  = robot.d;

    A1 = [-d/2, 0];
    A2 = [ d/2, 0];

    P = [x, y];

    % Left chain
    v1 = P - A1;
    r1 = norm(v1);

    alpha1 = atan2(v1(2), v1(1));
    c1 = (l1^2 + r1^2 - l3^2) / (2*l1*r1);
    c1 = max(min(c1, 1), -1);
    beta1 = acos(c1);

    theta1 = alpha1 + beta1;

    % Right chain
    v2 = P - A2;
    r2 = norm(v2);

    alpha2 = atan2(v2(2), v2(1));
    c2 = (l2^2 + r2^2 - l4^2) / (2*l2*r2);
    c2 = max(min(c2, 1), -1);
    beta2 = acos(c2);

    theta2 = alpha2 - beta2;

    theta = [theta1; theta2];
end
```

B. References

References

[1] GIU Parallel Kinematics and Kinetics, *Tutorial 8: Velocity Kinematics*.

[2] GIU Parallel Kinematics and Kinetics, *Tutorial 9: Static Forces and Virtual Work*.

[3] Northwestern University Mechatronics Wiki, "Design and Control of a Pantograph Robot," 2010. [Online]. Available: https://hades.mech.northwestern.edu/index.php/Design_and_Control_of_a_Pantograph_Robot. Accessed: May 15, 2026.

[4] M. D. Ratolikar and P. K. R., "Optimized design of 5R planar parallel mechanism aimed at gait-cycle of quadruped robots," *Journal of Vibroengineering*, 2021. doi: https://doi.org/10.21595/jve.2021.22131.